5. Users, Groups, sudo, and /etc/passwd

05. Users, Groups, sudo, and /etc/passwd

Phase: Phase 2 — Permissions and identity
Estimated time: 1–2 days
Slug: `05-users-groups`

Learning objectives

- Explain passwd/shadow/group files at a high level
- Use id, whoami, groups
- Apply least-privilege with sudo
- Create users/groups on a lab system (carefully)

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

Linux is a multi-user system. Each process runs as some user, and that identity decides which files it can touch and which operations it may perform. Knowing how users, groups and `sudo` work lets you give people just enough access, and avoid logging in as root for routine work, which is one of the biggest safety habits in administration.

Core concepts

A user has a numeric *UID*; the name is only a label. UID 0 is root, with nearly unlimited power. System accounts (daemons such as `www-data`) typically have low UIDs; on many distributions regular users start at 1000. Each user has a primary *GID* plus optional supplementary groups. Account data lives in text files.

```
/etc/passwd   name:x:UID:GID:comment:home:shell
/etc/group    group:x:GID:member1,member2
/etc/shadow   name:password-hash:...      (root only)
/etc/sudoers  who  where=(as whom)  commands
```

Command	Purpose
`id`	Show UID, GID and groups of current user
`whoami`	Print effective username
`groups`	List group names
`useradd` / `adduser`	Create a user (adduser is the friendlier Debian wrapper)
`usermod -aG GROUP USER`	Append user to a supplementary group
`passwd`	Change password
`su - USER`	Start a login shell as another user
`sudo CMD`	Run one command as root (or `-u USER`)
`visudo`	Safely edit sudoers

`sudo` asks for *your* password (not root's), checks the rules in `/etc/sudoers` and `/etc/sudoers.d/`, runs the command with elevated privileges and logs the event (often in `/var/log/auth.log` or the journal). On Ubuntu, members of the `sudo` group may run anything; on RHEL-like systems it is the `wheel` group.

Group membership changes take effect on *new* login sessions; run `newgrp` or log in again to see them.

Worked examples

Who am I:

```
id
# sample output: uid=1000(student) gid=1000(student)
groups=1000(student),27(sudo),100(users)
id -u
# sample output: 1000
id -un; id -gn
# sample output: student
# sample output: student
```

Look up accounts:

```
getent passwd student
# sample output: student:x:1000:1000:Student,,,:/home/student:/bin/bash
grep '^root:' /etc/passwd | cut -d: -f1,3,6,7
# sample output: root:0:/root:/bin/bash
```

Fields are separated by colons: field 3 is the UID.

Use sudo:

```
sudo whoami
# sample output: root
sudo -l | head -n 3
# sample output: Matching Defaults entries for student on box:
# sample output:     env_reset, mail_badpass, secure_path=...
```

Create a user and a group, add membership:

```
sudo groupadd project
sudo useradd -m -s /bin/bash -G project dana
sudo usermod -aG project student
getent group project
# sample output: project:x:1002:dana,student
```

Without `-a`, `usermod -G` would replace all supplementary groups.

Mini-lab

1. Run `id` and identify your UID, GID and groups.
2. Run `getent passwd $(id -un)` and name each of the seven fields.
3. Run `sudo -v` to validate sudo works (in a Docker container install it with `apt update && apt install -y sudo`).
4. Create a user: `sudo useradd -m -s /bin/bash lab5`; set a password with `sudo passwd lab5`.
5. Create a group: `sudo groupadd team5` and `sudo usermod -aG team5 lab5`; verify with `id lab5`.
6. Run `sudo -u lab5 id` and compare with your own `id`.
7. As lab5, run `touch /etc/x` and read the error; then `sudo touch /etc/x` as an allowed user and delete it.
8. Clean up: `sudo userdel -r lab5; sudo groupdel team5`.

Decision guide / failure modes

Use `sudo CMD` for single privileged actions with an audit trail; use `su -` or `sudo -i` only when you truly need a long root session; use dedicated service accounts (no login shell, `nologin`) for daemons. Grant groups rather than individual permissions, and prefer narrow sudo rules over `ALL`.

Common failures:

- `usermod -G` without `-a` silently removes other groups.
- Editing `/etc/sudoers` directly with a typo locks out sudo; always use `visudo`.
- Using `sudo` with redirection: `sudo echo x > /etc/f` fails because the redirection runs in your shell; use `echo x | sudo tee /etc/f`.
- Expecting new group rights without re-login.
- Running everything as root, so one typo or compromised script owns the machine.

Common mistakes

- Editing passwd by hand when `useradd` exists
- Sharing root password instead of sudo
- Leaving passwordless sudo on internet-facing boxes

Exercises

Work in `exercises/05-users-groups/`. Read that folder's `README.md` fully before coding.

```
cd exercises/05-users-groups
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement parse_uid(id_output). The input is a single line produced by the id command, for example "uid=1000(student) gid=1000(student) groups=1000(student),27(sudo)". Return the numeric user id as an int, taken from the "uid=" field (the digits right after "uid=", before the parenthesis or space). Ignore gid and groups. If there is no uid= field, return –1. The input may have leading or trailing whitespace. This is the same extraction you would do with grep -o or cut in a script.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/05-users-groups/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What is in `/etc/shadow` vs `/etc/passwd`?
2. What does `id` show?
3. Why prefer sudo over shared root?

Further reading

- https://man7.org/linux/man-pages/man5/passwd.5.html